JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LocalDate

java.lang.Object
    java.time.LocalDate

All Implemented Interfaces:

Serializable, Comparable<ChronoLocalDate>, ChronoLocalDate, Temporal, TemporalAccessor, TemporalAdjuster

---

```
public final class LocalDate
extends Object
implements Temporal, TemporalAdjuster, ChronoLocalDate, Serializable
```

A date without a time-zone in the ISO-8601 calendar system, such as `2007-12-03`.

`LocalDate` is an immutable date-time object that represents a date, often viewed as year-month-day. Other date fields, such as day-of-year, day-of-week and week-of-year, can also be accessed. For example, the value "2nd October 2007" can be stored in a `LocalDate`.

This class does not store or represent a time or time-zone. Instead, it is a description of the date, as used for birthdays. It cannot represent an instant on the time-line without additional information such as an offset or time-zone.

The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time. For most applications written today, the ISO-8601 rules are entirely suitable. However, any application that makes use of historical dates, and requires them to be accurate will find the ISO-8601 approach unsuitable.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final LocalDate	EPOCH	The epoch year LocalDate, '1970-01-01'.
static final LocalDate	MAX	The maximum supported LocalDate, '+999999999-12-31'.
static final LocalDate	MIN	The minimum supported LocalDate, '-999999999-01-01'.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
Temporal	adjustInto(Temporal temporal)	Adjusts the specified temporal object to have the same date as this object.
LocalDateTime	atStartOfDay()	Combines this date with the time of midnight to create a LocalDateTime at the start of this date.
ZonedDateTime	atStartOfDay(ZoneId zone)	Returns a zoned date-time from this date at the earliest valid time according to the rules in the time-zone.
LocalDateTime	atTime(int hour, int minute)	Combines this date with a time to create a LocalDateTime.

LocalDateTime	**atTime**(int hour, int minute, int second)	Combines this date with a time to create a LocalDateTime.
LocalDateTime	**atTime**(int hour, int minute, int second, int nanoOfSecond)	Combines this date with a time to create a LocalDateTime.
LocalDateTime	**atTime**(LocalTime time)	Combines this date with a time to create a LocalDateTime.
OffsetDateTime	**atTime**(OffsetTime time)	Combines this date with an offset time to create an OffsetDateTime.
int	**compareTo**(ChronoLocalDate other)	Compares this date to another date.
Stream<LocalDate>	**datesUntil**(LocalDate endExclusive)	Returns a sequential ordered stream of dates.
Stream<LocalDate>	**datesUntil**(LocalDate endExclusive, Period step)	Returns a sequential ordered stream of dates by given incremental step.
boolean	**equals**(Object obj)	Checks if this date is equal to another date.
String	**format**(DateTimeFormatter forma	Formats this date using the specified formatter.
static LocalDate	**from**(TemporalAccessor tempor	Obtains an instance of LocalDate from a temporal object.
int	**get**(TemporalField field)	Gets the value of the specified field from this date as an int.
IsoChronology	**getChronology**()	Gets the chronology of this date, which is the ISO calendar system.
int	**getDayOfMonth**()	Gets the day-of-month field.

DayOfWeek	**getDayOfWeek**()	Gets the day-of-week field, which is an enum DayOfWeek.
int	**getDayOfYear**()	Gets the day-of-year field.
IsoEra	**getEra**()	Gets the era applicable at this date.
long	**getLong** (TemporalField field)	Gets the value of the specified field from this date as a long.
Month	**getMonth**()	Gets the month-of-year field using the Month enum.
int	**getMonthValue**()	Gets the month-of-year field from 1 to 12.
int	**getYear**()	Gets the year field.
int	**hashCode**()	A hash code for this date.
boolean	**isAfter** (ChronoLocalDate other)	Checks if this date is after the specified date.
boolean	**isBefore** (ChronoLocalDate other)	Checks if this date is before the specified date.
boolean	**isEqual** (ChronoLocalDate other)	Checks if this date is equal to the specified date.
boolean	**isLeapYear**()	Checks if the year is a leap year, according to the ISO proleptic calendar system rules.
boolean	**isSupported** (TemporalField field)	Checks if the specified field is supported.
boolean	**isSupported** (TemporalUnit unit)	Checks if the specified unit is supported.

int	**lengthOfMonth**()	Returns the length of the month represented by this date.
int	**lengthOfYear**()	Returns the length of the year represented by this date.
LocalDate	**minus**(long amountToSubtract, TemporalUnit unit)	Returns a copy of this date with the specified amount subtracted.
LocalDate	**minus**(TemporalAmount amountTo	Returns a copy of this date with the specified amount subtracted.
LocalDate	**minusDays**(long daysToSubtract)	Returns a copy of this LocalDate with the specified number of days subtracted.
LocalDate	**minusMonths**(long monthsToSubtract)	Returns a copy of this LocalDate with the specified number of months subtracted.
LocalDate	**minusWeeks**(long weeksToSubtract)	Returns a copy of this LocalDate with the specified number of weeks subtracted.
LocalDate	**minusYears**(long yearsToSubtract)	Returns a copy of this LocalDate with the specified number of years subtracted.
static LocalDate	**now**()	Obtains the current date from the system clock in the default time-zone.
static LocalDate	**now**(Clock clock)	Obtains the current date from the specified clock.
static LocalDate	**now**(ZoneId zone)	Obtains the current date from the system clock in the specified time-zone.

static LocalDate	**of**(int year, int month, int dayOfMonth)	Obtains an instance of LocalDate from a year, month and day.
static LocalDate	**of**(int year, Month month, int dayOfMonth)	Obtains an instance of LocalDate from a year, month and day.
static LocalDate	**ofEpochDay** (long epochDay)	Obtains an instance of LocalDate from the epoch day count.
static LocalDate	**ofInstant** (Instant instant, ZoneId zone)	Obtains an instance of LocalDate from an Instant and zone ID.
static LocalDate	**ofYearDay**(int year, int dayOfYear)	Obtains an instance of LocalDate from a year and day-of-year.
static LocalDate	**parse** (CharSequence text)	Obtains an instance of LocalDate from a text string such as 2007-12-03.
static LocalDate	**parse** (CharSequence text, DateTimeFormatter format	Obtains an instance of LocalDate from a text string using a specific formatter.
LocalDate	**plus**(long amountToAdd, TemporalUnit unit)	Returns a copy of this date with the specified amount added.
LocalDate	**plus** (TemporalAmount amountTo	Returns a copy of this date with the specified amount added.
LocalDate	**plusDays** (long daysToAdd)	Returns a copy of this LocalDate with the specified number of days added.
LocalDate	**plusMonths** (long monthsToAdd)	Returns a copy of this LocalDate with the specified number of months added.

LocalDate	**plusWeeks**(long weeksToAdd)	Returns a copy of this LocalDate with the specified number of weeks added.
LocalDate	**plusYears**(long yearsToAdd)	Returns a copy of this LocalDate with the specified number of years added.
<R> R	**query**(TemporalQuery<R> query)	Queries this date using the specified query.
ValueRange	**range**(TemporalField field)	Gets the range of valid values for the specified field.
long	**toEpochSecond**(LocalTime time, ZoneOffset offset)	Converts this LocalDate to the number of seconds since the epoch of 1970-01-01T00:00:00Z.
String	**toString**()	Outputs this date as a String, such as 2007-12-03.
Period	**until**(ChronoLocalDate endDate	Calculates the period between this date and another date as a Period.
long	**until**(Temporal endExclusive, TemporalUnit unit)	Calculates the amount of time until another date in terms of the specified unit.
LocalDate	**with**(TemporalAdjuster adjust	Returns an adjusted copy of this date.
LocalDate		

	with(TemporalField field, long newValue)	Returns a copy of this date with the specified field set to a new value.
LocalDate	**withDayOfMonth**(int dayOfMonth)	Returns a copy of this LocalDate with the day-of-month altered.
LocalDate	**withDayOfYear**(int dayOfYear)	Returns a copy of this LocalDate with the day-of-year altered.
LocalDate	**withMonth**(int month)	Returns a copy of this LocalDate with the month-of-year altered.
LocalDate	**withYear**(int year)	Returns a copy of this LocalDate with the year altered.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface ChronoLocalDate

toEpochDay

Field Details

MIN

```
public static final LocalDate MIN
```

The minimum supported LocalDate, '-999999999-01-01'. This could be used by an application as a "far past" date.

MAX

```
public static final LocalDate MAX
```

The maximum supported LocalDate, '+999999999-12-31'. This could be used by an application as a "far future" date.

EPOCH

```
public static final LocalDate EPOCH
```

The epoch year `LocalDate`, '1970-01-01'.

Since:

9

Method Details

now

```
public static LocalDate now()
```

Obtains the current date from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

the current date using the system clock and default time-zone, not null

now

```
public static LocalDate now(ZoneId zone)
```

Obtains the current date from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current date using the system clock, not null

now

```
public static LocalDate now(Clock clock)
```

Obtains the current date from the specified clock.

This will query the specified clock to obtain the current date - today. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

`clock` - the clock to use, not null

Returns:

the current date, not null

of

```
public static LocalDate of(int year,
                           Month month,
                           int dayOfMonth)
```

Obtains an instance of `LocalDate` from a year, month and day.

This returns a `LocalDate` with the specified year, month and day-of-month. The day must be valid for the year and month, otherwise an exception will be thrown.

Parameters:

`year` - the year to represent, from MIN_YEAR to MAX_YEAR

`month` - the month-of-year to represent, not null

`dayOfMonth` - the day-of-month to represent, from 1 to 31

Returns:

the local date, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

of

```
public static LocalDate of(int year,
                           int month,
                           int dayOfMonth)
```

Obtains an instance of `LocalDate` from a year, month and day.

This returns a `LocalDate` with the specified year, month and day-of-month. The day must be valid for the year and month, otherwise an exception will be thrown.

Parameters:

`year` - the year to represent, from MIN_YEAR to MAX_YEAR

`month` - the month-of-year to represent, from 1 (January) to 12 (December)

`dayOfMonth` - the day-of-month to represent, from 1 to 31

Returns:

the local date, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

ofYearDay

```
public static LocalDate ofYearDay(int year,
                                  int dayOfYear)
```

Obtains an instance of `LocalDate` from a year and day-of-year.

This returns a `LocalDate` with the specified year and day-of-year. The day-of-year must be valid for the year, otherwise an exception will be thrown.

Parameters:

`year` - the year to represent, from MIN_YEAR to MAX_YEAR

`dayOfYear` - the day-of-year to represent, from 1 to 366

Returns:

the local date, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-year is invalid for the year

ofInstant

```
public static LocalDate ofInstant(Instant instant,
                                  ZoneId zone)
```

Obtains an instance of `LocalDate` from an `Instant` and zone ID.

This creates a local date based on the specified instant. First, the offset from UTC/Greenwich is obtained using the zone ID and instant, which is simple as there is only one valid offset for each instant. Then, the instant and offset are used to calculate the local date.

Parameters:

`instant` - the instant to create the date from, not null

`zone` - the time-zone, which may be an offset, not null

Returns:

the local date, not null

Throws:

`DateTimeException` - if the result exceeds the supported range

Since:

9

ofEpochDay

```
public static LocalDate ofEpochDay(long epochDay)
```

Obtains an instance of `LocalDate` from the epoch day count.

This returns a `LocalDate` with the specified epoch-day. The `EPOCH_DAY` is a simple incrementing count of days where day 0 is 1970-01-01. Negative numbers represent earlier days.

Parameters:

`epochDay` - the Epoch Day to convert, based on the epoch 1970-01-01

Returns:

the local date, not null

Throws:

`DateTimeException` - if the epoch day exceeds the supported date range

from

```
public static LocalDate from(TemporalAccessor temporal)
```

Obtains an instance of `LocalDate` from a temporal object.

This obtains a local date based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `LocalDate`.

The conversion uses the `TemporalQueries.localDate()` query, which relies on extracting the `EPOCH_DAY` field.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `LocalDate::from`.

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the local date, not null

Throws:

`DateTimeException` - if unable to convert to a `LocalDate`

parse

```
public static LocalDate parse(CharSequence text)
```

Obtains an instance of `LocalDate` from a text string such as `2007-12-03`.

The string must represent a valid date and is parsed using `DateTimeFormatter.ISO_LOCAL_DATE`.

Parameters:

`text` - the text to parse such as "2007-12-03", not null

Returns:

the parsed local date, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

parse

```
public static LocalDate parse(CharSequence text,
                              DateTimeFormatter formatter)
```

Obtains an instance of `LocalDate` from a text string using a specific formatter.

The text is parsed using the formatter, returning a date.

Parameters:

`text` - the text to parse, not null

`formatter` - the formatter to use, not null

Returns:

the parsed local date, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this date can be queried for the specified field. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

If the field is a ChronoField then the query is implemented here. The supported fields are:

- DAY_OF_WEEK
- ALIGNED_DAY_OF_WEEK_IN_MONTH
- ALIGNED_DAY_OF_WEEK_IN_YEAR
- DAY_OF_MONTH
- DAY_OF_YEAR
- EPOCH_DAY
- ALIGNED_WEEK_OF_MONTH
- ALIGNED_WEEK_OF_YEAR
- MONTH_OF_YEAR
- PROLEPTIC_MONTH
- YEAR_OF_ERA
- YEAR
- ERA

All other ChronoField instances will return false.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.isSupportedBy(TemporalAccessor) passing this as the argument. Whether the field is supported is determined by the field.

Specified by:

isSupported in interface ChronoLocalDate

Specified by:

isSupported in interface TemporalAccessor

Parameters:

field - the field to check, null returns false

Returns:

true if the field is supported on this date, false if not

isSupported

```
public boolean isSupported(TemporalUnit unit)
```

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this date. If false, then calling the plus(long, TemporalUnit) and minus methods will throw an exception.

If the unit is a ChronoUnit then the query is implemented here. The supported units are:

- DAYS

- WEEKS
- MONTHS
- YEARS
- DECADES
- CENTURIES
- MILLENNIA
- ERAS

All other ChronoUnit instances will return false.

If the unit is not a ChronoUnit, then the result of this method is obtained by invoking TemporalUnit.isSupportedBy(Temporal) passing this as the argument. Whether the unit is supported is determined by the unit.

Specified by:

isSupported in interface ChronoLocalDate

Specified by:

isSupported in interface Temporal

Parameters:

unit - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This date is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return appropriate range instances. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.rangeRefinedBy(TemporalAccessor) passing this as the argument. Whether the range can be obtained is determined by the field.

Specified by:

range in interface TemporalAccessor

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this date as an `int`.

This queries this date for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this date, except `EPOCH_DAY` and `PROLEPTIC_MONTH` which are too large to fit in an `int` and throw an `UnsupportedTemporalTypeException`. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

`UnsupportedTemporalTypeException` - if the field is not supported or the range of values exceeds an `int`

`ArithmeticException` - if numeric overflow occurs

getLong

`public long getLong(TemporalField field)`

Gets the value of the specified field from this date as a `long`.

This queries this date for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this date. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.getFrom(TemporalAccessor)` passing `this` as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

getChronology

`public IsoChronology getChronology()`

Gets the chronology of this date, which is the ISO calendar system.

The `Chronology` represents the calendar system in use. The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time.

Specified by:

`getChronology` in interface `ChronoLocalDate`

Returns:

the ISO chronology, not null

getEra

```
public IsoEra getEra()
```

Gets the era applicable at this date.

The official ISO-8601 standard does not define eras, however `IsoChronology` does. It defines two eras, 'CE' from year one onwards and 'BCE' from year zero backwards. Since dates before the Julian-Gregorian cutover are not in line with history, the cutover between 'BCE' and 'CE' is also not aligned with the commonly used eras, often referred to using 'BC' and 'AD'.

Users of this class should typically ignore this method as it exists primarily to fulfill the `ChronoLocalDate` contract where it is necessary to support the Japanese calendar system.

Specified by:

`getEra` in interface `ChronoLocalDate`

Returns:

the IsoEra applicable at this date, not null

getYear

```
public int getYear()
```

Gets the year field.

This method returns the primitive `int` value for the year.

The year returned by this method is proleptic as per `get(YEAR)`. To obtain the year-of-era, use `get(YEAR_OF_ERA)`.

Returns:

the year, from MIN_YEAR to MAX_YEAR

getMonthValue

```
public int getMonthValue()
```

Gets the month-of-year field from 1 to 12.

This method returns the month as an `int` from 1 to 12. Application code is frequently clearer if the enum `Month` is used by calling `getMonth()`.

Returns:

the month-of-year, from 1 to 12

See Also:

`getMonth()`

getMonth

`public Month getMonth()`

Gets the month-of-year field using the `Month` enum.

This method returns the enum `Month` for the month. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Returns:

the month-of-year, not null

See Also:

`getMonthValue()`

getDayOfMonth

`public int getDayOfMonth()`

Gets the day-of-month field.

This method returns the primitive `int` value for the day-of-month.

Returns:

the day-of-month, from 1 to 31

getDayOfYear

`public int getDayOfYear()`

Gets the day-of-year field.

This method returns the primitive `int` value for the day-of-year.

Returns:

the day-of-year, from 1 to 365, or 366 in a leap year

getDayOfWeek

`public DayOfWeek getDayOfWeek()`

Gets the day-of-week field, which is an enum `DayOfWeek`.

This method returns the enum `DayOfWeek` for the day-of-week. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Additional information can be obtained from the `DayOfWeek`. This includes textual names of the values.

Returns:

the day-of-week, not null

isLeapYear

```
public boolean isLeapYear()
```

Checks if the year is a leap year, according to the ISO proleptic calendar system rules.

This method applies the current rules for leap years across the whole time-line. In general, a year is a leap year if it is divisible by four without remainder. However, years divisible by 100, are not leap years, with the exception of years divisible by 400 which are.

For example, 1904 is a leap year it is divisible by 4. 1900 was not a leap year as
it is divisible by 100, however 2000 was a leap year as it is divisible by 400.

The calculation is proleptic - applying the same rules into the far future and far past. This is historically inaccurate, but is correct for the ISO-8601 standard.

Specified by:

`isLeapYear` in interface `ChronoLocalDate`

Returns:

true if the year is leap, false otherwise

lengthOfMonth

```
public int lengthOfMonth()
```

Returns the length of the month represented by this date.

This returns the length of the month in days. For example, a date in January would return 31.

Specified by:

`lengthOfMonth` in interface `ChronoLocalDate`

Returns:

the length of the month in days

lengthOfYear

`public int lengthOfYear()`

Returns the length of the year represented by this date.

This returns the length of the year in days, either 365 or 366.

Specified by:

`lengthOfYear` in interface `ChronoLocalDate`

Returns:

366 if the year is leap, 365 otherwise

with

`public LocalDate with(TemporalAdjuster adjuster)`

Returns an adjusted copy of this date.

This returns a `LocalDate`, based on this one, with the date adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month.

A selection of common adjustments is provided in `TemporalAdjusters`. These include finding the "last day of the month" and "next Wednesday". Key date-time classes also implement the `TemporalAdjuster` interface, such as `Month` and `MonthDay`. The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

For example this code returns a date on the last day of July:

```
import static java.time.Month.*;
import static java.time.temporal.TemporalAdjusters.*;

result = localDate.with(JULY).with(lastDayOfMonth());
```

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `ChronoLocalDate`

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

a `LocalDate` based on `this` with the adjustment made, not null

Throws:

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

with

```
public LocalDate with(TemporalField field,
                      long newValue)
```

Returns a copy of this date with the specified field set to a new value.

This returns a `LocalDate`, based on this one, with the value for the specified field changed. This can be used to change any supported field, such as the year, month or day-of-month. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

In some cases, changing the specified field can cause the resulting date to become invalid, such as changing the month from 31st January to February would make the day-of-month invalid. In cases like this, the field is responsible for resolving the date. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

If the field is a `ChronoField` then the adjustment is implemented here. The supported fields behave as follows:

- `DAY_OF_WEEK` - Returns a `LocalDate` with the specified day-of-week. The date is adjusted up to 6 days forward or backward within the boundary of a Monday to Sunday week.
- `ALIGNED_DAY_OF_WEEK_IN_MONTH` - Returns a `LocalDate` with the specified aligned-day-of-week. The date is adjusted to the specified month-based aligned-day-of-week. Aligned weeks are counted such that the first week of a given month starts on the first day of that month. This may cause the date to be moved up to 6 days into the following month.
- `ALIGNED_DAY_OF_WEEK_IN_YEAR` - Returns a `LocalDate` with the specified aligned-day-of-week. The date is adjusted to the specified year-based aligned-day-of-week. Aligned weeks are counted such that the first week of a given year starts on the first day of that year. This may cause the date to be moved up to 6 days into the following year.

- `DAY_OF_MONTH` - Returns a `LocalDate` with the specified day-of-month. The month and year will be unchanged. If the day-of-month is invalid for the year and month, then a `DateTimeException` is thrown.
- `DAY_OF_YEAR` - Returns a `LocalDate` with the specified day-of-year. The year will be unchanged. If the day-of-year is invalid for the year, then a `DateTimeException` is thrown.
- `EPOCH_DAY` - Returns a `LocalDate` with the specified epoch-day. This completely replaces the date and is equivalent to `ofEpochDay(long)`.
- `ALIGNED_WEEK_OF_MONTH` - Returns a `LocalDate` with the specified aligned-week-of-month. Aligned weeks are counted such that the first week of a given month starts on the first day of that month. This adjustment moves the date in whole week chunks to match the specified week. The result will have the same day-of-week as this date. This may cause the date to be moved into the following month.
- `ALIGNED_WEEK_OF_YEAR` - Returns a `LocalDate` with the specified aligned-week-of-year. Aligned weeks are counted such that the first week of a given year starts on the first day of that year. This adjustment moves the date in whole week chunks to match the specified week. The result will have the same day-of-week as this date. This may cause the date to be moved into the following year.
- `MONTH_OF_YEAR` - Returns a `LocalDate` with the specified month-of-year. The year will be unchanged. The day-of-month will also be unchanged, unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `PROLEPTIC_MONTH` - Returns a `LocalDate` with the specified proleptic-month. The day-of-month will be unchanged, unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `YEAR_OF_ERA` - Returns a `LocalDate` with the specified year-of-era. The era and month will be unchanged. The day-of-month will also be unchanged, unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `YEAR` - Returns a `LocalDate` with the specified year. The month will be unchanged. The day-of-month will also be unchanged, unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `ERA` - Returns a `LocalDate` with the specified era. The year-of-era and month will be unchanged. The day-of-month will also be unchanged, unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.

In all cases, if the new value is outside the valid range of values for the field then a `DateTimeException` will be thrown.

All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `ChronoLocalDate`

Specified by:

`with` in interface `Temporal`

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

a `LocalDate` based on `this` with the specified field set, not null

Throws:

`DateTimeException` - if the field cannot be set

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

withYear

```
public LocalDate withYear(int year)
```

Returns a copy of this `LocalDate` with the year altered.

If the day-of-month is invalid for the year, it will be changed to the last valid day of the month.

This instance is immutable and unaffected by this method call.

Parameters:

`year` - the year to set in the result, from MIN_YEAR to MAX_YEAR

Returns:

a `LocalDate` based on this date with the requested year, not null

Throws:

DateTimeException - if the year value is invalid

withMonth

```
public LocalDate withMonth(int month)
```

Returns a copy of this `LocalDate` with the month-of-year altered.

If the day-of-month is invalid for the year, it will be changed to the last valid day of the month.

This instance is immutable and unaffected by this method call.

Parameters:

`month` - the month-of-year to set in the result, from 1 (January) to 12 (December)

Returns:

a `LocalDate` based on this date with the requested month, not null

Throws:

DateTimeException - if the month-of-year value is invalid

withDayOfMonth

```
public LocalDate withDayOfMonth(int dayOfMonth)
```

Returns a copy of this `LocalDate` with the day-of-month altered.

If the resulting date is invalid, an exception is thrown.

This instance is immutable and unaffected by this method call.

Parameters:

`dayOfMonth` - the day-of-month to set in the result, from 1 to 28-31

Returns:

a `LocalDate` based on this date with the requested day, not null

Throws:

DateTimeException - if the day-of-month value is invalid, or if the day-of-month is invalid for the month-year

withDayOfYear

```
public LocalDate withDayOfYear(int dayOfYear)
```

Returns a copy of this `LocalDate` with the day-of-year altered.

If the resulting date is invalid, an exception is thrown.

This instance is immutable and unaffected by this method call.

Parameters:

`dayOfYear` - the day-of-year to set in the result, from 1 to 365-366

Returns:

a `LocalDate` based on this date with the requested day, not null

Throws:

`DateTimeException` - if the day-of-year value is invalid, or if the day-of-year is invalid for the year

plus

```
public LocalDate plus(TemporalAmount amountToAdd)
```

Returns a copy of this date with the specified amount added.

This returns a `LocalDate`, based on this one, with the specified amount added. The amount is typically `Period` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `ChronoLocalDate`

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount to add, not null

Returns:

a `LocalDate` based on this date with the addition made, not null

Throws:

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

plus

```
public LocalDate plus(long amountToAdd,
                      TemporalUnit unit)
```

Returns a copy of this date with the specified amount added.

This returns a `LocalDate`, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

In some cases, adding the amount can cause the resulting date to become invalid. For example, adding one month to 31st January would result in 31st February. In cases like this, the unit is responsible for resolving the date. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

If the field is a `ChronoUnit` then the addition is implemented here. The supported fields behave as follows:

- `DAYS` - Returns a `LocalDate` with the specified number of days added. This is equivalent to `plusDays(long)`.
- `WEEKS` - Returns a `LocalDate` with the specified number of weeks added. This is equivalent to `plusWeeks(long)` and uses a 7 day week.
- `MONTHS` - Returns a `LocalDate` with the specified number of months added. This is equivalent to `plusMonths(long)`. The day-of-month will be unchanged unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `YEARS` - Returns a `LocalDate` with the specified number of years added. This is equivalent to `plusYears(long)`. The day-of-month will be unchanged unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `DECADES` - Returns a `LocalDate` with the specified number of decades added. This is equivalent to calling `plusYears(long)` with the amount multiplied by 10. The day-of-month will be unchanged unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `CENTURIES` - Returns a `LocalDate` with the specified number of centuries added. This is equivalent to calling `plusYears(long)` with the amount multiplied by 100. The day-of-month will be unchanged unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.
- `MILLENNIA` - Returns a `LocalDate` with the specified number of millennia added. This is equivalent to calling `plusYears(long)` with the amount

multiplied by 1,000. The day-of-month will be unchanged unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.

- `ERAS` - Returns a `LocalDate` with the specified number of eras added. Only two eras are supported so the amount must be one, zero or minus one. If the amount is non-zero then the year is changed such that the year-of-era is unchanged. The day-of-month will be unchanged unless it would be invalid for the new month and year. In that case, the day-of-month is adjusted to the maximum valid value for the new month and year.

All other `ChronoUnit` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `ChronoLocalDate`

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount of the unit to add to the result, may be negative

`unit` - the unit of the amount to add, not null

Returns:

a `LocalDate` based on this date with the specified amount added, not null

Throws:

`DateTimeException` - if the addition cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

plusYears

```
public LocalDate plusYears(long yearsToAdd)
```

Returns a copy of this `LocalDate` with the specified number of years added.

This method adds the specified amount to the years field in three steps:

1. Add the input years to the year field

2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2008-02-29 (leap year) plus one year would result in the invalid date 2009-02-29 (standard year). Instead of returning an invalid result, the last valid day of the month, 2009-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`yearsToAdd` - the years to add, may be negative

Returns:

a `LocalDate` based on this date with the years added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusMonths

```
public LocalDate plusMonths(long monthsToAdd)
```

Returns a copy of this `LocalDate` with the specified number of months added.

This method adds the specified amount to the months field in three steps:

1. Add the input months to the month-of-year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2007-03-31 plus one month would result in the invalid date 2007-04-31. Instead of returning an invalid result, the last valid day of the month, 2007-04-30, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`monthsToAdd` - the months to add, may be negative

Returns:

a `LocalDate` based on this date with the months added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusWeeks

```
public LocalDate plusWeeks(long weeksToAdd)
```

Returns a copy of this `LocalDate` with the specified number of weeks added.

This method adds the specified amount in weeks to the days field incrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2008-12-31 plus one week would result in 2009-01-07.

This instance is immutable and unaffected by this method call.

Parameters:

`weeksToAdd` - the weeks to add, may be negative

Returns:

a `LocalDate` based on this date with the weeks added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusDays

```
public LocalDate plusDays(long daysToAdd)
```

Returns a copy of this `LocalDate` with the specified number of days added.

This method adds the specified amount to the days field incrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2008-12-31 plus one day would result in 2009-01-01.

This instance is immutable and unaffected by this method call.

Parameters:

`daysToAdd` - the days to add, may be negative

Returns:

a `LocalDate` based on this date with the days added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minus

```
public LocalDate minus(TemporalAmount amountToSubtract)
```

Returns a copy of this date with the specified amount subtracted.

This returns a `LocalDate`, based on this one, with the specified amount subtracted. The amount is typically `Period` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `ChronoLocalDate`

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

a `LocalDate` based on this date with the subtraction made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public LocalDate minus(long amountToSubtract,
                       TemporalUnit unit)
```

Returns a copy of this date with the specified amount subtracted.

This returns a `LocalDate`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `ChronoLocalDate`

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

a `LocalDate` based on this date with the specified amount subtracted, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

minusYears

```
public LocalDate minusYears(long yearsToSubtract)
```

Returns a copy of this `LocalDate` with the specified number of years subtracted.

This method subtracts the specified amount from the years field in three steps:

1. Subtract the input years from the year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2008-02-29 (leap year) minus one year would result in the invalid date 2007-02-29 (standard year). Instead of returning an invalid result, the last valid day of the month, 2007-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`yearsToSubtract` - the years to subtract, may be negative

Returns:

a `LocalDate` based on this date with the years subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusMonths

```
public LocalDate minusMonths(long monthsToSubtract)
```

Returns a copy of this `LocalDate` with the specified number of months subtracted.

This method subtracts the specified amount from the months field in three steps:

1. Subtract the input months from the month-of-year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2007-03-31 minus one month would result in the invalid date 2007-02-31. Instead of returning an invalid result, the last valid day of the month, 2007-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`monthsToSubtract` - the months to subtract, may be negative

Returns:

a `LocalDate` based on this date with the months subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusWeeks

```
public LocalDate minusWeeks(long weeksToSubtract)
```

Returns a copy of this `LocalDate` with the specified number of weeks subtracted.

This method subtracts the specified amount in weeks from the days field decrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2009-01-07 minus one week would result in 2008-12-31.

This instance is immutable and unaffected by this method call.

Parameters:

`weeksToSubtract` - the weeks to subtract, may be negative

Returns:

a `LocalDate` based on this date with the weeks subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

minusDays

```
public LocalDate minusDays(long daysToSubtract)
```

Returns a copy of this `LocalDate` with the specified number of days subtracted.

This method subtracts the specified amount from the days field decrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2009-01-01 minus one day would result in 2008-12-31.

This instance is immutable and unaffected by this method call.

Parameters:

`daysToSubtract` - the days to subtract, may be negative

Returns:

a `LocalDate` based on this date with the days subtracted, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this date using the specified query.

This queries this date using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `ChronoLocalDate`

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

DateTimeException - if unable to query (defined by the query)

ArithmeticException - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have the same date as this object.

This returns a temporal object of the same observable type as the input with the date changed to be the same as this.

The adjustment is equivalent to using Temporal.with(TemporalField, long) passing ChronoField.EPOCH_DAY as the field.

In most cases, it is clearer to reverse the calling pattern by using Temporal.with(TemporalAdjuster):

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisLocalDate.adjustInto(temporal);
temporal = temporal.with(thisLocalDate);
```

This instance is immutable and unaffected by this method call.

Specified by:

adjustInto in interface ChronoLocalDate

Specified by:

adjustInto in interface TemporalAdjuster

Parameters:

temporal - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

DateTimeException - if unable to make the adjustment

ArithmeticException - if numeric overflow occurs

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another date in terms of the specified unit.

This calculates the amount of time between two `LocalDate` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified date. The result will be negative if the end is before the start. The `Temporal` passed to this method is converted to a `LocalDate` using `from(TemporalAccessor)`. For example, the amount in days between two dates can be calculated using `startDate.until(endDate, DAYS)`.

The calculation returns a whole number, representing the number of complete units between the two dates. For example, the amount in months between 2012-06-15 and 2012-08-14 will only be one month as it is one day short of two months.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MONTHS);
amount = MONTHS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` are supported. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `ChronoLocalDate`

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive` - the end date, exclusive, which is converted to a `LocalDate`, not null

`unit` - the unit to measure the amount in, not null

Returns:

the amount of time between this date and the end date

Throws:

`DateTimeException` - if the amount cannot be calculated, or the end temporal cannot be converted to a `LocalDate`

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

until

`public Period until(ChronoLocalDate endDateExclusive)`

Calculates the period between this date and another date as a `Period`.

This calculates the period between two dates in terms of years, months and days. The start and end points are `this` and the specified date. The result will be negative if the end is before the start. The negative sign will be the same in each of year, month and day.

The calculation is performed using the ISO calendar system. If necessary, the input date will be converted to ISO.

The start date is included, but the end date is not. The period is calculated by removing complete months, then calculating the remaining number of days, adjusting to ensure that both have the same sign. The number of months is then normalized into years and months based on a 12 month year. A month is considered to be complete if the end day-of-month is greater than or equal to the start day-of-month. For example, from `2010-01-15` to `2011-03-18` is "1 year, 2 months and 3 days".

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `Period.between(LocalDate, LocalDate)`:

```
// these two lines are equivalent
period = start.until(end);
period = Period.between(start, end);
```

The choice should be made based on which makes the code more readable.

Specified by:

`until` in interface `ChronoLocalDate`

Parameters:

`endDateExclusive` - the end date, exclusive, which may be in any chronology, not null

Returns:

the period between this date and the end date, not null

datesUntil

`public Stream<LocalDate> datesUntil(LocalDate endExclusive)`

Returns a sequential ordered stream of dates. The returned stream starts from this date (inclusive) and goes to `endExclusive` (exclusive) by an incremental step of 1 day.

This method is equivalent to `datesUntil(endExclusive, Period.ofDays(1))`.

Parameters:

`endExclusive` - the end date, exclusive, not null

Returns:

a sequential `Stream` for the range of `LocalDate` values

Throws:

`IllegalArgumentException` - if end date is before this date

Since:

9

datesUntil

```
public Stream<LocalDate> datesUntil(LocalDate endExclusive,
                                    Period step)
```

Returns a sequential ordered stream of dates by given incremental step. The returned stream starts from this date (inclusive) and goes to `endExclusive` (exclusive).

The n-th date which appears in the stream is equal to `this.plus(step.multipliedBy(n))` (but the result of step multiplication never overflows). For example, if this date is `2015-01-31`, the end date is `2015-05-01` and the step is 1 month, then the stream contains `2015-01-31`, `2015-02-28`, `2015-03-31`, and `2015-04-30`.

Parameters:

`endExclusive` - the end date, exclusive, not null

`step` - the non-zero, non-negative `Period` which represents the step.

Returns:

a sequential `Stream` for the range of `LocalDate` values

Throws:

`IllegalArgumentException` - if step is zero, or `step.getDays()` and `step.toTotalMonths()` have opposite sign, or end date is before this date and step is positive, or end date is after this date and step is negative

Since:

9

format

```
public String format(DateTimeFormatter formatter)
```

Formats this date using the specified formatter.

This date will be passed to the formatter to produce a string.

Specified by:

format in interface ChronoLocalDate

Parameters:

formatter - the formatter to use, not null

Returns:

the formatted date string, not null

Throws:

DateTimeException - if an error occurs during printing

atTime

```
public LocalDateTime atTime(LocalTime time)
```

Combines this date with a time to create a LocalDateTime.

This returns a LocalDateTime formed from this date at the specified time. All possible combinations of date and time are valid.

Specified by:

atTime in interface ChronoLocalDate

Parameters:

time - the time to combine with, not null

Returns:

the local date-time formed from this date and the specified time, not null

atTime

```
public LocalDateTime atTime(int hour,
                            int minute)
```

Combines this date with a time to create a LocalDateTime.

This returns a LocalDateTime formed from this date at the specified hour and minute. The seconds and nanosecond fields will be set to zero. The individual time fields must be within their valid range. All possible combinations of date and time are valid.

Parameters:

`hour` - the hour-of-day to use, from 0 to 23

`minute` - the minute-of-hour to use, from 0 to 59

Returns:

the local date-time formed from this date and the specified time, not null

Throws:

`DateTimeException` - if the value of any field is out of range

atTime

```
public LocalDateTime atTime(int hour,
                            int minute,
                            int second)
```

Combines this date with a time to create a `LocalDateTime`.

This returns a `LocalDateTime` formed from this date at the specified hour, minute and second. The nanosecond field will be set to zero. The individual time fields must be within their valid range. All possible combinations of date and time are valid.

Parameters:

`hour` - the hour-of-day to use, from 0 to 23

`minute` - the minute-of-hour to use, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

Returns:

the local date-time formed from this date and the specified time, not null

Throws:

`DateTimeException` - if the value of any field is out of range

atTime

```
public LocalDateTime atTime(int hour,
                            int minute,
                            int second,
                            int nanoOfSecond)
```

Combines this date with a time to create a `LocalDateTime`.

This returns a `LocalDateTime` formed from this date at the specified hour, minute, second and nanosecond. The individual time fields must be within their valid range. All possible combinations of date and time are valid.

Parameters:

`hour` - the hour-of-day to use, from 0 to 23

`minute` - the minute-of-hour to use, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

`nanoOfSecond` - the nano-of-second to represent, from 0 to 999,999,999

Returns:

the local date-time formed from this date and the specified time, not null

Throws:

`DateTimeException` - if the value of any field is out of range

atTime

```
public OffsetDateTime atTime(OffsetTime time)
```

Combines this date with an offset time to create an `OffsetDateTime`.

This returns an `OffsetDateTime` formed from this date at the specified time. All possible combinations of date and time are valid.

Parameters:

`time` - the time to combine with, not null

Returns:

the offset date-time formed from this date and the specified time, not null

atStartOfDay

```
public LocalDateTime atStartOfDay()
```

Combines this date with the time of midnight to create a `LocalDateTime` at the start of this date.

This returns a `LocalDateTime` formed from this date at the time of midnight, 00:00, at the start of this date.

Returns:

the local date-time of midnight at the start of this date, not null

atStartOfDay

```
public ZonedDateTime atStartOfDay(ZoneId zone)
```

Returns a zoned date-time from this date at the earliest valid time according to the rules in the time-zone.

Time-zone rules, such as daylight savings, mean that not every local date-time is valid for the specified zone, thus the local date-time may not be midnight.

In most cases, there is only one valid offset for a local date-time. In the case of an overlap, there are two valid offsets, and the earlier one is used, corresponding to the first occurrence of midnight on the date. In the case of a gap, the zoned date-time will represent the instant just after the gap.

If the zone ID is a `ZoneOffset`, then the result always has a time of midnight.

To convert to a specific time in a given time-zone call `atTime(LocalTime)` followed by `LocalDateTime.atZone(ZoneId)`.

Parameters:

`zone` - the zone ID to use, not null

Returns:

the zoned date-time formed from this date and the earliest valid time for the zone, not null

toEpochSecond

```
public long toEpochSecond(LocalTime time,
                          ZoneOffset offset)
```

Converts this `LocalDate` to the number of seconds since the epoch of 1970-01-01T00:00:00Z.

This combines this local date with the specified time and offset to calculate the epoch-second value, which is the number of elapsed seconds from 1970-01-01T00:00:00Z. Instants on the time-line after the epoch are positive, earlier are negative.

Parameters:

`time` - the local time, not null

`offset` - the zone offset, not null

Returns:

the number of seconds since the epoch of 1970-01-01T00:00:00Z, may be negative

Since:

9

compareTo

```
public int compareTo(ChronoLocalDate other)
```

Compares this date to another date.

The comparison is primarily based on the date, from earliest to latest. It is "consistent with equals", as defined by `Comparable`.

If all the dates being compared are instances of `LocalDate`, then the comparison will be entirely based on the date. If some dates being compared are in different chronologies, then the chronology is also considered, see `ChronoLocalDate.compareTo(ChronoLocalDate)`.

Specified by:

`compareTo` in interface `ChronoLocalDate`

Specified by:

`compareTo` in interface `Comparable<ChronoLocalDate>`

Parameters:

`other` - the other date to compare to, not null

Returns:

the comparator value, that is the comparison of this local date with the `other` local date and this chronology with the `other` chronology, in order, returning the first non-zero result, and otherwise returning zero

See Also:

`isBefore(ChronoLocalDate)`, `isAfter(ChronoLocalDate)`

isAfter

```
public boolean isAfter(ChronoLocalDate other)
```

Checks if this date is after the specified date.

This checks to see if this date represents a point on the local time-line after the other date.

```
LocalDate a = LocalDate.of(2012, 6, 30);
LocalDate b = LocalDate.of(2012, 7, 1);
a.isAfter(b) == false
a.isAfter(a) == false
b.isAfter(a) == true
```

This method only considers the position of the two dates on the local time-line. It does not take into account the chronology, or calendar system. This is different from the comparison in `compareTo(ChronoLocalDate)`, but is the same approach as `ChronoLocalDate.timeLineOrder()`.

Specified by:

`isAfter` in interface `ChronoLocalDate`

Parameters:

`other` - the other date to compare to, not null

Returns:

true if this date is after the specified date

isBefore

`public boolean isBefore(ChronoLocalDate other)`

Checks if this date is before the specified date.

This checks to see if this date represents a point on the local time-line before the other date.

```
LocalDate a = LocalDate.of(2012, 6, 30);
LocalDate b = LocalDate.of(2012, 7, 1);
a.isBefore(b) == true
a.isBefore(a) == false
b.isBefore(a) == false
```

This method only considers the position of the two dates on the local time-line. It does not take into account the chronology, or calendar system. This is different from the comparison in `compareTo(ChronoLocalDate)`, but is the same approach as `ChronoLocalDate.timeLineOrder()`.

Specified by:

`isBefore` in interface `ChronoLocalDate`

Parameters:

`other` - the other date to compare to, not null

Returns:

true if this date is before the specified date

isEqual

`public boolean isEqual(ChronoLocalDate other)`

Checks if this date is equal to the specified date.

This checks to see if this date represents the same point on the local time-line as the other date.

```
LocalDate a = LocalDate.of(2012, 6, 30);
LocalDate b = LocalDate.of(2012, 7, 1);
a.isEqual(b) == false
a.isEqual(a) == true
b.isEqual(a) == false
```

This method only considers the position of the two dates on the local time-line. It does not take into account the chronology, or calendar system. This is different from the comparison in `compareTo(ChronoLocalDate)` but is the same approach as `ChronoLocalDate.timeLineOrder()`.

Specified by:

`isEqual` in interface `ChronoLocalDate`

Parameters:

`other` - the other date to compare to, not null

Returns:

true if this date is equal to the specified date

equals

`public boolean equals(Object obj)`

Checks if this date is equal to another date.

Compares this `LocalDate` with another ensuring that the date is the same.

Only objects of type `LocalDate` are compared, other types return false. To compare the dates of two `TemporalAccessor` instances, including dates in two different chronologies, use `ChronoField.EPOCH_DAY` as a comparator.

Specified by:

`equals` in interface `ChronoLocalDate`

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other date

See Also:

`Object.hashCode()`, `HashMap`

hashCode

`public int hashCode()`

A hash code for this date.

Specified by:

`hashCode` in interface `ChronoLocalDate`

Overrides:

hashCode in class Object

Returns:

a suitable hash code

See Also:

Object.equals(java.lang.Object),
System.identityHashCode(Object)

toString

```
public String toString()
```

Outputs this date as a String, such as 2007-12-03.

The output will be in the ISO-8601 format uuuu-MM-dd.

Specified by:

toString in interface ChronoLocalDate

Overrides:

toString in class Object

Returns:

a string representation of this date, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.